

Ayudantía 2 x

Maquetación Web Responsiva & CRP

< HTML + CSS > → [CRP] → {Interfaz}



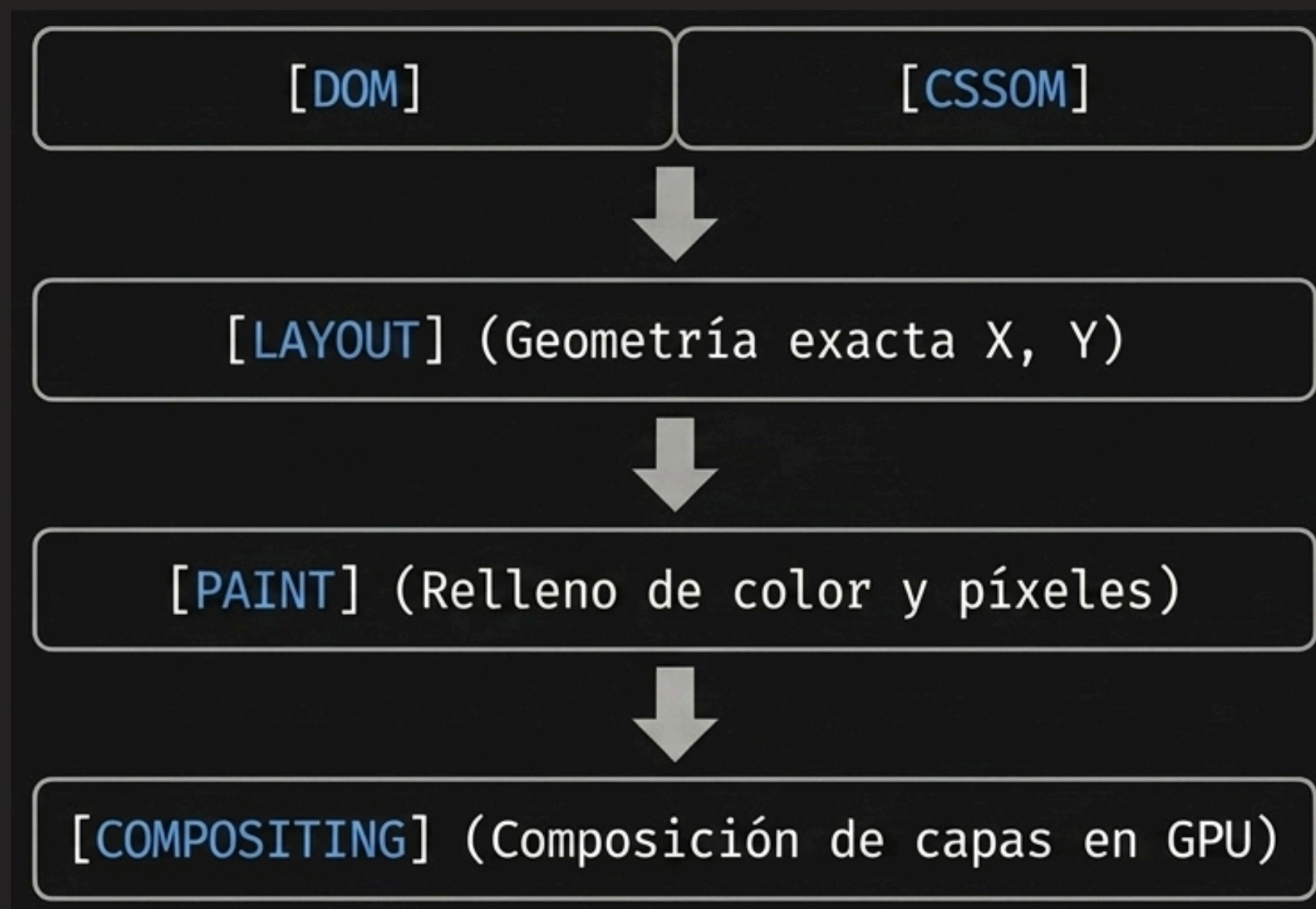
CRITICAL RENDERING PATH

- La secuencia de pasos que sigue el Browser para transformar código fuente (HTML, CSS, JS) en píxeles interactivos en pantalla.
- Se procesa en milisegundos por el motor de renderizado aislado en cada pestaña.
- **El objetivo:** Optimizar este camino para mostrar el primer contenido lo antes posible

[Servidor] —(Bytes)→ [Navegador: CRP] —(Píxeles)→ [Pantalla]

ETAPAS DEL CRP

- Estructura jerárquica del HTML y árbol de reglas de CSS.
- Cálculo de la geometría exacta (Reflow).
- Relleno de píxeles
- Ensamblaje y superposición de capas directamente en la GPU

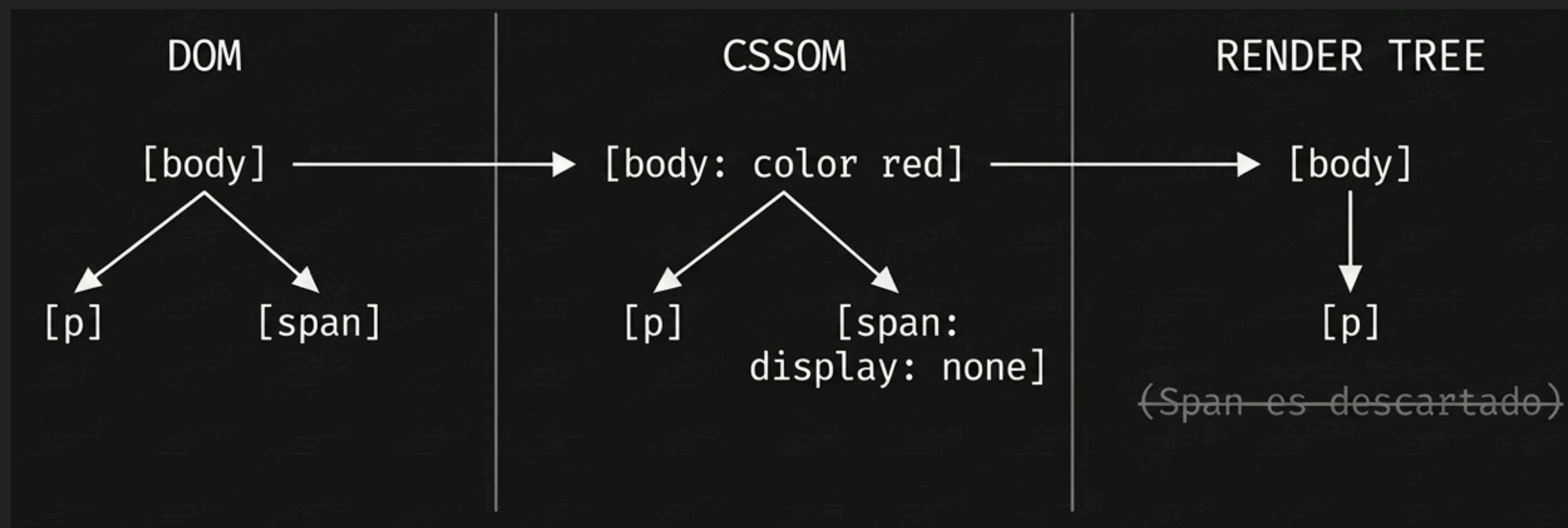




RENDER TREE

Fusiona la estructura (DOM) con el diseño (CSSOM) excluyendo lo invisible.

Un nodo con `display: none` NO entra en el Render Tree. Si tiene la propiedad `visibility: hidden` SÍ lo hace (reserva espacio



COSTO COMPUTACIONAL

Para lograr animaciones fluidas, se debe limitar la manipulación al paso de **COMPOSITING**. Modificar geometría obliga a recalcular toda la página.

MODIFICACIÓN CSS	DISPARA LAS FASES...	IMPACTO RENDIMIENTO
width, height, margin	→ Layout + Paint + Comp	[ALTO - CPU]
color, background	→ Paint + Comp	[MEDIO - CPU]
transform, opacity	→ Solo Compositing	[BAJO - GPU]

CSSOM Y EL BLOQUEO DE RENDERIZADO

El Browser frena el dibujado hasta construir el CSSOM al 100% para evitar parpadeos sin estilo (FOUC - Flash of Unstyled Content).



```
1  <!-- Carga crítica. Bloqua el renderizado (Alto impacto) -->
2  <link rel="stylesheet" href="base.css">
3
4  <!-- Desactiva el bloqueo de render en Desktop (Bajo prioridad) -->
5  <link rel="stylesheet" href="mobile.css" media="(max-width: 600px)">
```

Mitigación: Dividir el CSS y usar el atributo media para que el navegador descargue en baja prioridad lo que no aplique a la pantalla actual



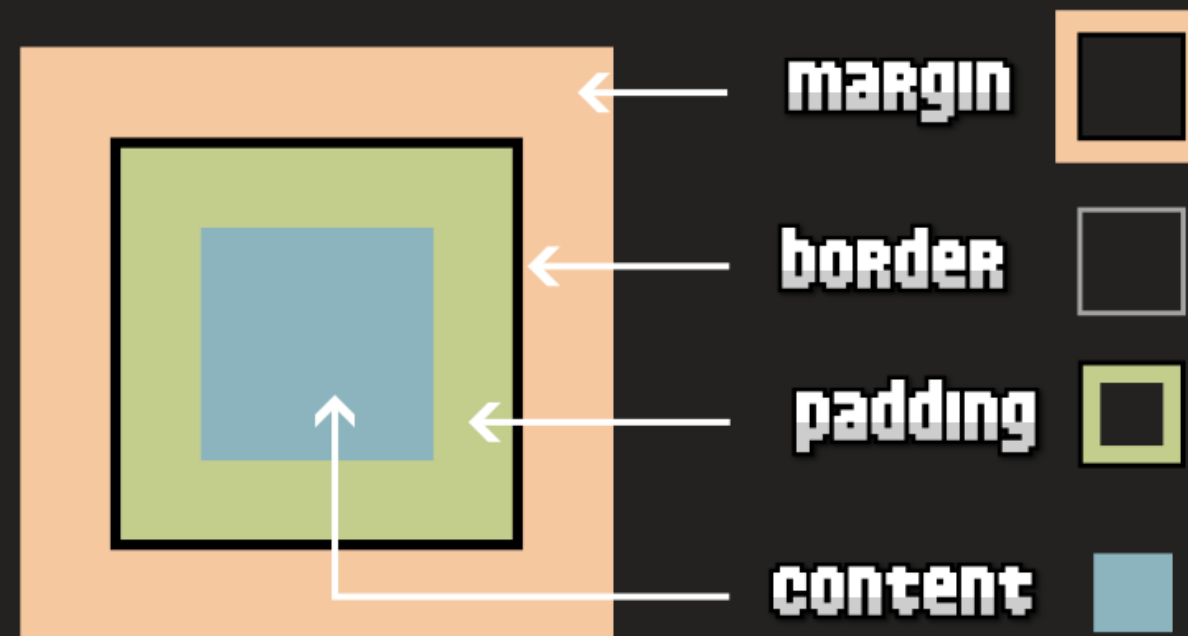
BOX MODEL

CONTENT:

Espacio ocupado por el contenido

PADDING:

Espacio protector.
Adquiere el color de fondo



BORDER:

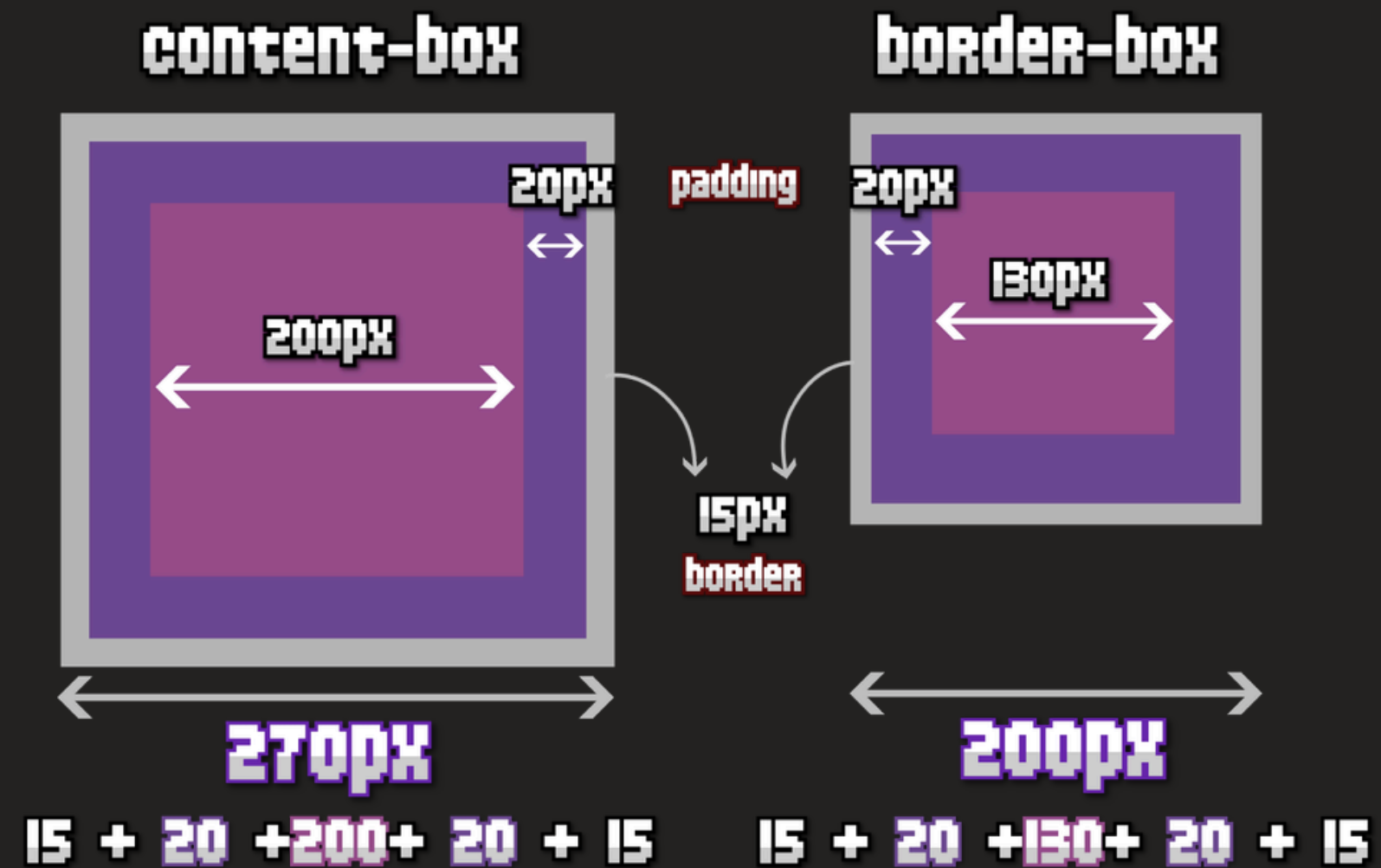
Límite físico de la caja.

MARGIN:

Espacio transparente que emuja a cajas vecinas

DIMENSIÓN: BORDER-BOX VS CONTENT-BOX

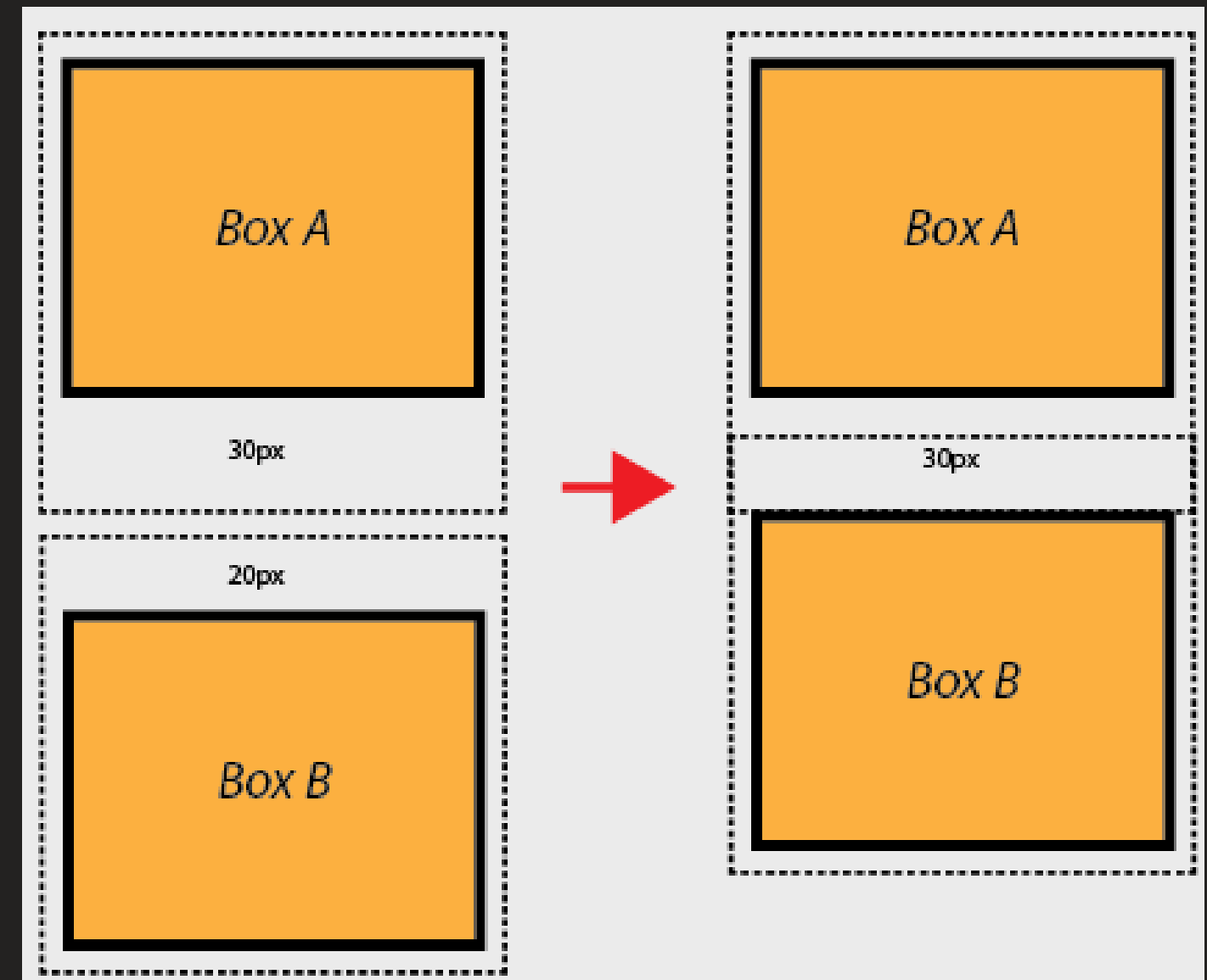
Estandariza el cálculo predecible de las dimensiones



MARGIN COLLAPSE

Situación que ocurre en estos modelos de maquetación básica, donde los márgenes de dos elementos se colapsan, ocupando el mismo tamaño una sola vez.

OBS: Esto no aplica a Flex y Grid.



FLEXBOX

Es un modelo de diseño web que ayuda a crear estructuras de **una sólo dimensión**.

Permite organizar los elementos dentro de un contenedor de manera más flexible, estableciendo filas o columnas dentro de un contenedor.





FLEXBOX: POPIEDADES

DISPLAY: FLEX:

Activa el Flexbox en un contendor.

JUSTIFY-CONTENT:

Alinea los elementos a lo largo del eje principal.

FLEX-WRAP:

Controla si los elementos se ajustan en varias lineas

FLEX-DIRECTION:

Define la dirección principal del eje principal.

ALIGN-ITEMS:

Aliena los elementos a lo largo del eje secundario.



FLEXBOX: PROPIEDADES HIJOS

FLEX-GROW:

Permite que un elemento crezca según el espacio disponible

FLEX-SHRINK:

Permite que un elemento se reduzca si no hay suficiente espacio

FLEX-BASIS:

Define el tamaño inicial de un elemento antes de distribuir el espacio.

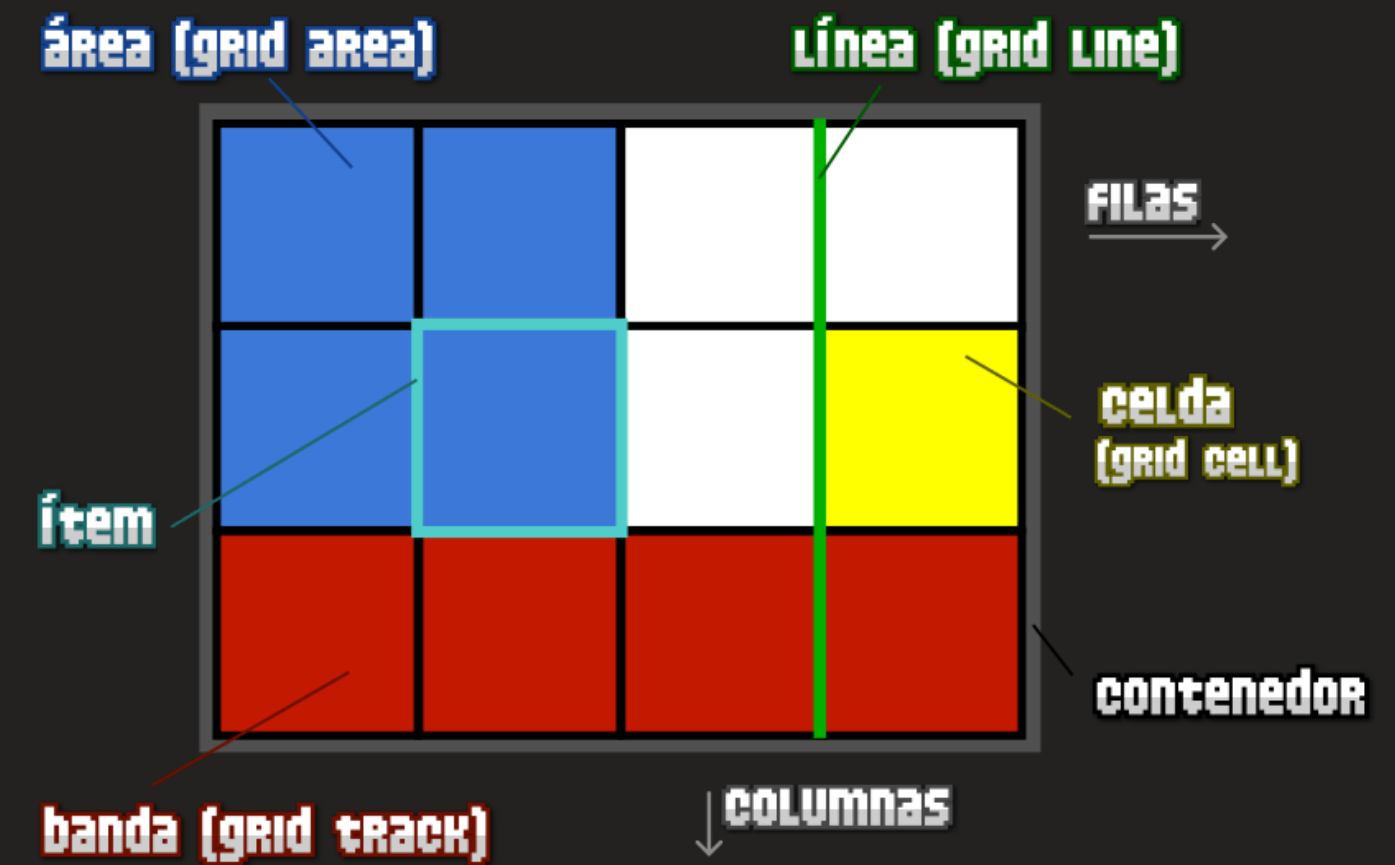




GRID

Es una herramienta que ayuda a organizar elementos en una página web como si fuera una cuadrícula. Permite colocar cajas en filas y columnas.

Es ideal para hacer estructuras como galerías de imágenes, secciones de una página o dashboards, ya que todo se alinea de forma ordenada y flexible.





GRID: PROPIEDADES

DISPLAY: GRID:

Activa la grilla en un contenedor.

GRID-TEMPLTE-COLUMNS:

Define el número y tamaño de columnas.

GRID-TEMPLATE-ROWS:

Define el número y tamaño de filas.

GAP:

Espacio entre elementos.

GRID-TEMPLATE-AREA:

Indica nombre y posición de cada área.



GRID: PROPIEDADES HIJOS

GRID-ROW:

Define en qué fila inicia y termina un elemento.

GRID-COLUMN:

Define en qué columna inicia y termina un elemento.



FLEXBOX VS GRID

DIRECCIÓN

Flexbox organiza elementos en una sola dimensión (fila o columna). Mientras que Grid permite organizar contenido en dos dimensiones.

USO COMÚN

Flexbox es excelente para componentes como menús y alineaciones dentro de contenedores pequeños. Grid se usa para construir layouts completos, como páginas web y dashboards.

ADAPTABILIDAD

Flexbox es más flexible para elementos dinámicos que cambian de tamaño o posición. En cambio, Grid es ideal para estructuras fijas.

EJEMPLO

Flexbox facilita la creación de barras de navegación o centrar elementos fácilmente. Grid permite construir interfaces completas con filas y columnas bien definidas.



PÁGINAS RESPONSIVAS

- No usar pixeles estáticos. Usar dinámicos: rem, % y fr
- Diseñar estilos para pantallas pequeñas reduce la carga en procesadores móviles.
- La Media Queries usan min-width para inyectar complejidad escalonada

```

1  .columna {
2      width: 100%;
3  }
4
5  @media (min-width: 768px) {
6      .columna {
7          width: 50%;
8      }
9  }
    
```



```

1  <meta name="viewport" content="width=device-width, initial-scale=1.0">
    
```

Ayudantía 2 CRP Render Tree CSS Flexbox Grid Maquetación Responsividad Actividad x



ACTIVIDAD



- Aplicar Box Model & Reset a los elementos.
- Aplicar Flexbox para alinear internamente las tarjetas.
- Aplicar una Grilla responsiva con Grid.
- Responder pregunta reflexiva en Canvas.





RECURSOS

- [Manz.dev](#) y [bootcamp](#)
- [Midudev](#)
- [Flexbox](#), [Grid](#) y [Planillas Grid](#)

